

# Lecture 1: Course Overview and LLM Basics

## Theoretical Foundations in Modern Machine Learning

Allen Liu | CSCI-GA.3033-141

September 8, 2026

The course studies simple mathematical models that inform modern model training. We follow the modern training pipeline. Tentative schedule: LLM fundamentals (meetings 1–2), pretraining (3–5), post-training (6–9), and test-time scaling (10–12), followed by two flex meetings. Course policies are on the [course website](#) and in its linked syllabus.

## 1 A language model as a next-token predictor

Text is represented by a sequence of *tokens* from a vocabulary  $\mathcal{V}$ . A token can represent a word, part of a word, punctuation, or another symbol. Tokenization and the vocabulary depend on the model. Given a token sequence  $x_{<t} = (x_1, \dots, x_{t-1})$ , a language model with parameters  $\theta$  outputs a vector of *logits*  $\ell_\theta(x_{<t}) \in \mathbb{R}^{|\mathcal{V}|}$ . Applying softmax gives the next-token distribution:

$$p_\theta(v \mid x_{<t}) = \frac{\exp(\ell_\theta(x_{<t})_v)}{\sum_{u \in \mathcal{V}} \exp(\ell_\theta(x_{<t})_u)}, \quad v \in \mathcal{V}.$$

Thus logits are unnormalized log probabilities: their differences determine probability ratios, and adding the same constant to every logit changes nothing.

**Autoregressive generation.** Starting from a prompt, sample a next token, append it to the sequence, and repeat until an end marker or a length limit. The probability of a generated response  $y_{1:T}$ , conditional on a prompt  $x$ , is

$$p_\theta(y_{1:T} \mid x) = \prod_{t=1}^T p_\theta(y_t \mid x, y_{<t}).$$

The above is simply using the chain rule for conditional probabilities. The model parameters  $\theta$  remain fixed, but the context  $x, y_{<t}$  grows with each generated token.

## 2 Context limits and copying

A model accepts a bounded amount of context. For longer histories, consider the following explicit abstraction: the system retains only the most recent  $k$  tokens, with no other persistent state or access to older tokens. We call  $k$  the context length of the model. Thus, for a sequence  $X_1, \dots, X_n$  generated by the model, a token  $X_t$  is independent of tokens more than  $k$  in the past, given the most recent  $k$  tokens. Formally, for  $t > k + 1$ ,

$$X_t \perp\!\!\!\perp X_{1:t-k-1} \mid X_{t-k:t-1}.$$

This is an order- $k$  Markov property. In the usual  $n$ -gram convention, which counts the predicted token as well, it corresponds to a  $(k + 1)$ -gram model.

**Note: why arbitrarily long strings cannot always be copied.** Suppose the input contains a string of tokens  $s \in \{0, 1\}^n$ ,  $n > k$ , followed by a fixed instruction to copy it. Choose two strings that differ in their first token and agree everywhere else. Their visible suffixes are identical when the response begins, so the model must use the same distribution for the first output token on both inputs. The correct first tokens differ. Hence it cannot copy both strings with certainty.

Of course, modern LLMs can get around this limitation by leveraging various external tools, e.g., programmatically writing the text to a file and then copying it.

### 3 Chat, reasoning, and tools

So far, we have a very simple abstraction of LLMs as next-token predictors. It turns out that this simple abstraction is sufficient to describe and understand many of the incredible capabilities and behaviors we see in practice, including chat, reasoning, and coding/tool use.

**Chat.** A conversation is serialized as tokens identifying speakers and message boundaries. After an assistant response, a user message can be appended and generation resumed. A typical template is `<user> question </user> <assistant> answer </assistant>`. The actual control tokens and end-of-turn conventions vary between models; the tags here are illustrative. See the [Hugging Face chat-template documentation](#) for concrete examples.

Here, `<user>`, `</user>`, `<assistant>`, `</assistant>` are usually special tokens in the model's vocabulary. Basically, when the user sends a message, the model sees the entire conversation history up to `<assistant>` and then generates autoregressively until it generates `</assistant>`. At this point, the assistant's "turn" ends and the message is sent to the user and the user's next turn begins.

**Reasoning traces.** How do language models think and reason about complex problems? Many of you have probably heard of "chain-of-thought". This allows a model to generate intermediate reasoning tokens before an answer. Typically, the model will produce a sequence of the form `<user>What is 2+2? </user><assistant><think> Let me think about this... This is a very simple math question, so I should just answer. </think> The answer is 4. </assistant>`

These tokens are generated autoregressively. The model learns when to generate `<think>` tokens and to place its reasoning/thought process within a `<think> </think>` block. Usually, the message sent back to the user is the part of the message outside of the `<think> </think>` block. Training can encourage this behavior.

Nowadays, both chat templates and templates for thinking tokens are quite simple to train into the model via supervised fine-tuning (SFT).

**Tool use.** We now often talk about models being agentic, meaning that they can interact with their environment and use tools to accomplish tasks. How do agents (e.g., coding agents) do this? A coding agent pairs the model with an external program called a *harness* or *orchestrator*. This is ordinary code, for example a Python program.

Basically, the model outputs a tool call, which is an exactly formatted string that says that it wants to, e.g., read/write/edit/grep a file. The harness parses the tool call programmatically (so the tool call must be structured and formatted exactly correctly) and returns the result of the tool call back to the model. If the model outputs a "read" tool call, the harness parses this call and returns the contents of the file the model wants to read. These contents are then appended to the conversation history. We can consider each interaction via a tool call as a "turn".

A tool call looks something like:

```
<assistant>
<tool_call>
{"name": "read_file", "arguments": {"path": "settings.py"}}
</tool_call>
</assistant>
```

Note that these tool call tokens are generated autoregressively by the model – they must be formatted exactly correctly, otherwise we end up with what is known as a "bad tool call", one that the harness cannot parse. Because of this requirement, smaller models often have trouble with a fairly high frequency of bad tool calls.

See the [Hugging Face tool-use documentation](#) for a concrete implementation.

**Example (illustrative tool names and syntax):**

```
User: In settings.py, change X from 2 to 3.
Model -> harness: read_file({"path": "settings.py"})
Harness -> model [tool result]: "X = 2\n"
Model -> harness: edit_file({"path": "settings.py",
    "old": "X = 2", "new": "X = 3"})
Harness -> model [tool result]: "Replaced 1 occurrence."
Model -> user: Changed X from 2 to 3.
```

## 4 The training and inference pipeline

The modern training pipeline has two main stages: pretraining and post-training. Even after we have trained a model, there is a third axis that is very important as well: test-time scaling.

**Pretraining.** Pretraining teaches broad patterns and skills from a large corpus of everything, e.g., "all of the internet" (Wikipedia, code, stories, textbooks, etc.). Pretraining follows a very simple mathematical objective: next-token prediction. We want to minimize the negative log likelihood of next-token prediction across the entire corpus. For a model with parameters  $\theta$ , document  $y_{1:n}$  and a context of  $k$ , the loss is

$$L(\theta; y_{1:n}) = - \sum_{i=1}^n \log p_{\theta}(y_i \mid y_{i-k:i-1}).$$

The prefix for  $i \leq k$  is padded with special padding tokens. Training averages losses over batches and updates  $\theta$  using a gradient descent-type algorithm.

Pretraining teaches general patterns and knowledge, but the behavior of pretrained models is not very refined. For example, if you interact with a pretrained model and ask it a question, it will often respond with another question. This is because if you train on, say, all of the internet, then usually questions are followed by other questions in a list of exercises. Pretraining is all about scale and efficiency; frontier-scale pretraining runs often take months. Thus the main questions include how to allocate parameters, data, and compute, and how to choose and mix training data, especially given that you often only get one chance at the main run.

**Post-training.** While pretraining teaches general patterns and knowledge, post-training is where behavior gets refined to give us usable and useful models. Within post-training, there are two phases.

*Continued Pretraining/Mid-training.* Here we continue training with the same objective as pretraining (next-token prediction) on a more targeted mixture, such as code or mathematical reasoning data. This is often where formatting, thinking and chat templates are taught. Here we also teach more precise reasoning patterns, such as how to think about complex math problems. The data for this stage may come from carefully scraped and curated web data, previous generation models, or human-expert demonstrations.

**Reinforcement learning.** All of the previous stages involve training our model with next-token prediction to mimic data we've curated from external sources.

In RL, we train a model on its own outputs. Basically we reward the good outputs (update the model weights to make them more likely) and penalize the bad outputs and then iterate. More formally, for a prompt  $x$ , we generate responses  $y_1, \dots, y_n \sim p_{\theta}(\cdot \mid x)$ , called *rollouts*, and score them with a reward  $r(x, y_i)$ . A reward can be a correctness check, such as passing tests, or a learned prediction of human preference. The basic objective is

$$J(\theta) = \mathbb{E}_x \mathbb{E}_{y \sim p_{\theta}(\cdot \mid x)} [r(x, y)].$$

Roughly, all RL methods use these scores to update the model to favor better responses. Recent practical developments involve improved ways to control stability, and address intended vs. unintended learned behaviors.

**Test-time scaling.** Even after a model is trained, there is another important axis: how to make use of the model at test time. Here the model weights  $\theta$  stay fixed, but we can spend more computation on answering a particular question. This is what we mean by *test-time scaling*.

Two natural ways to achieve this are to force the model to think longer before answering (e.g., forcing the model to check its own work on a math problem, keep thinking, etc.), or to instead sample many parallel attempts from the same model and then have some method of picking the best one (best-of- $n$ ). For example, we could generate several candidate solutions, run tests on each, and select one that passes.

Another important direction is coordinating multiple agents that are each powered by the same model: they can try different approaches or work on different parts of a problem in parallel. For a recent, very timely example of coordinating agents, see [OpenAI’s account of the Hugging Face incident](#). The main questions are how to leverage the extra computation at test time, e.g., how to allocate a limited budget across individual attempts, parallel threads, and verification.